Password Cracking tools

Tool Name	Primary Purpose	Supported Protocols/Services	Unique Features	Use Cases	When to Use
Hydra	Brute-force attack for various protocols	SSH, FTP, HTTP, Telnet, LDAP, RDP, SMB, SMTP, etc.	High speed, multi-threaded, customizable command set	General-purpose brute-forcing for a variety of services	Use when testing a broad range of protocols and services
Ncrack	Network authentication cracking tool	SSH, RDP, FTP, HTTP(S), POP3(S), VNC, SMB, etc.	Focused on performance, timing options, efficient brute-forcing	Network service testing, auditing large-scale environments	Use for high-performance and large-scale authentication testing
Medusa	Modular brute-forcing tool	SSH, FTP, HTTP(S), IMAP, LDAP, MSSQL, SMB, VNC, etc.	Highly modular, parallelized, supports many protocols	Fast brute-forcing of multiple hosts simultaneously	Use when you need multi-threaded attacks on different protocols
Patator	Multi-purpose brute-forcing tool	SSH, FTP, HTTP(S), SMB, RDP, DNS, SNMP, etc.	Modular design, detailed logging, supports many scenarios	Complex brute-forcing scenarios, logging-intensive tasks	Use when stability and detailed logging are needed

Crowbar	Brute-force SSH key and RDP logins	SSH (key-based), RDP, VNC, OpenVPN	Specializes in key-based and password-less brute-forcing	Penetration testing for SSH key-based authentication	Use for SSH key-based or RDP/VNC brute-forcing
THC-pptp-bruter	Brute-force PPTP VPN authentication	PPTP VPN	Fast brute-forcing of PPTP VPN CHAP authentication	Testing PPTP VPN configurations	Use when testing or auditing PPTP VPN security
BruteSpray	Automated post-Nmap brute-forcing	SSH, FTP, RDP, SMB, etc. (based on Nmap results)	Integrates with Nmap, uses multiple brute-force tools	Automating brute-forcing after initial port scanning	Use for automating brute-force attacks on discovered services
John the Ripper	Password hash cracking tool	Local password files, password hashes (NTLM, MD5, etc.)	Optimized for password hashes, GPU support	Offline password cracking and dictionary attacks	Use for cracking password hashes from captured data
Hashcat	Advanced password hash cracking tool	Hashes (MD5, SHA1, SHA256, NTLM, etc.)	GPU acceleration, supports rule-based attacks	High-speed password hash cracking	Use for GPU-based hash cracking scenarios

Detailed Comparison Analysis

- **Hydra vs. Medusa vs. Ncrack:**
  - **Hydra** is ideal for general-purpose brute-forcing because it supports a **wide range of protocols** and is highly customizable.
  - **Medusa** is best when you need **multi-threaded brute-forcing on multiple hosts** simultaneously and want a modular approach.
  - **Ncrack** is focused on **performance and efficiency** and is preferred for **large-scale network testing**.
- **Patator vs. Crowbar:**
  - **Patator** is a versatile option for **stable and detailed brute-forcing** across multiple protocols.
  - **Crowbar** excels in **key-based and password-less authentication brute-forcing**, which makes it unique for scenarios where traditional username-password attacks fail.
- **THC-pptp-bruter vs. BruteSpray:**
  - **THC-pptp-bruter** is highly specialized for **PPTP VPN** brute-forcing, making it less useful outside of this niche.
  - **BruteSpray** is an excellent tool for **automated brute-forcing** after discovering open ports using Nmap, ideal for **post-scan automation**.
- **John the Ripper vs. Hashcat:**

- **John the Ripper** is preferred for **CPU-based** cracking of **simple hashes** and is easier to configure.
- **Hashcat** is optimized for **GPU-based hash cracking** and can handle complex attacks with its **rule-based engine**.

When to Choose Each Tool:

1. **Use Hydra** when you need a quick, configurable tool for a variety of protocols.
2. **Use Ncrack** for high-performance brute-forcing in large environments.
3. **Use Medusa** for fast parallelized attacks on multiple hosts or protocols.
4. **Use Patator** when stability, logging, and complex scenarios are a priority.
5. **Use Crowbar** for SSH key-based or password-less logins (SSH, RDP).
6. **Use THC-pptp-bruter** for testing PPTP VPN configurations.
7. **Use BruteSpray** for automating brute-forcing after an Nmap scan.
8. **Use John the Ripper** for traditional offline hash cracking.
9. **Use Hashcat** for GPU-accelerated, rule-based hash cracking.

HYDRA

"THC-Hydra," is a popular password-cracking tool. It’s primarily used for brute-forcing login credentials on network services. Let’s start with the basics:

History and Development

Hydra was created by the The Hacker's Choice (THC), a group known for developing various network security tools. Its initial release was around twenty-oh-four (2004). It quickly became a go-to tool for ethical hackers and security researchers due to its flexibility and support for numerous protocols.

Use Cases

Hydra is mainly used to:

Brute-force credentials on various protocols, such as HTTP, FTP, SSH, Telnet, SMTP, SNMP, and many others.

Test the strength of passwords for network services.

Conduct penetration testing on web applications and network services.

Automated login attempts for research purposes.

Supported Protocols and Services

Hydra can target over fifty (50) different services, including:

SSH

FTP

SMTP

IMAP

HTTP/HTTPS (forms and basic authentication)

VNC

MySQL

Telnet

And many more...

How Hydra Works

Hydra uses dictionary-based or brute-force attacks. Here’s a step-by-step of what happens internally:

Connection Initiation: Hydra initiates a connection to the target service using a specific protocol.

Credential Attempt: It tries a combination of usernames and passwords from a provided wordlist.

Response Analysis: Hydra checks the server's response to see if the login was successful.

Success/Failure Handling: If successful, it logs the credentials. If not, it continues trying until the list is exhausted or the user stops it.

Basic Command Syntax

The basic command syntax for Hydra looks like this:

css

```
hydra [OPTIONS] [PROTOCOL://TARGET] [OPTIONS]
```

Examples of Hydra Commands

FTP Brute Force Attack:

css

```
hydra -l admin -P /path/to/password/list.txt ftp://192.168.1.100
```

-l: Specifies the username.

-P: Points to the password list file.

ftp://192.168.1.100: The target protocol and IP address.

SSH Brute Force Attack:

css

```
hydra -L /path/to/user/list.txt -P /path/to/password/list.txt ssh://192.168.1.100
```

-L: Specifies a file containing a list of usernames.

HTTP Form Brute Force:

css

```
hydra -l admin -P /path/to/password/list.txt 192.168.1.100 http-post-form "/login.php:user=^USER^&pass=^PASS^:Invalid login"
```

Targets an HTTP POST form with a specific structure.

Detailed Options Explained

-s [PORT]: Specify a non-standard port for the service.

-t [THREADS]: Define the number of threads to speed up the attack.

-V: Verbose mode, to show each attempt in real-time.

-f: Stop after the first valid credential is found.

Advanced Features

Parallel Attacks: Hydra supports multi-threading, making it faster than many single-threaded brute-force tools.

Flexible Module System: You can add support for new services by writing your own modules.

Supports HTTPS and Proxies: Hydra can target HTTPS websites and can also work through a proxy, allowing for greater anonymity.

Considerations and Limitations

Rate-Limiting: Many modern servers implement rate-limiting, making Hydra less effective.

Account Lockout: Some systems lock accounts after a few failed attempts, which can make brute-forcing more noticeable.

Wordlist Dependency: The success of Hydra highly depends on the quality and size of the wordlist.

Hydra vs. Other Tools

Compared to tools like Medusa and Ncrack, Hydra is more flexible and supports a wider range of services, but it might be slightly slower in certain scenarios.

Hydra Cheat Sheet

Installation

```
sudo apt install hydra
```

```
sudo snap install seclists
```

Basic Usage

Command	Description
`hydra <target> <service>`	Brute-force a single host
`hydra -h`	Show help message
`hydra <service> -U`	Show service module usage

Target specification

Option	Description
`<target> <service>`	Target host and service
`<service>://<host>`	Target host and service
`-s <port>`	Target port
`-M <file>`	Load targets from a file

Common services

Service	Description
`http[s]-{head|get|post}`	Basic HTTP(S) authentication
`http[s]-{get|post}-form`	HTTP(S) login form
`ftp[s]`	FTP login
`ssh`	SSH login
`mongodb`	MongoDB
`mysql`	MySQL
`postgres`	PostgreSQL
`redis`	Redis

Use `hydra -h` to see a full list of supported services.

Username and password specification

Flag	Description
`-l <username>`	Login with username
`-L <file>`	Load usernames from a file
`-p <password>`	Try password
`-P <file>`	Load passwords from a file
`-C <file>`	Load usernames and passwords from a file ("login:password")

Brute force password generation

Option	Description
-x 3:5:a	Generate passwords of 3 to 5 lowercase letters
-x 5:8:A1	Generate passwords of 5 to 8 uppercase letters and numbers

Output

Option	Descripton
`-o <file>`	Write found login/password combinations to file
`-b <format>`	Switch between `text`, `json` or `jsonv1`
`hydra -v`	Verbose mode
`hydra -V`	Show all attempts

Performance

Flag	Description
`-u`	Loop around users, not passwords
`-f`	Exit after first match

Examples

```
hydra -l admin -P passwords.txt <target> http-get
/admin

hydra -l admin -P passwords.txt <target>
https-post-form
"/login.php:username=^USER^&password=^PASS^:F=incorr
ect"

hydra -L users.txt -P passwords.txt -u -f
ssh://<target> -t 4

hydra -l admin -P passwords.txt ftp://<target>
```

Personalized wordlists

Command	Description
`sudo apt install cupp`	Install CUPP
`cupp -i`	Create a custom wordlist interactively

Medusa

History and Development

Medusa is an open-source, high-performance, parallel, and modular brute-forcing tool. It was developed as an alternative to Hydra, aiming to offer more speed and scalability. Medusa's development started around twenty-oh-five (2005) and has been maintained by security researchers and contributors. It’s particularly valued for its modular architecture, which allows developers to easily add new services or protocols. This flexibility is one of the reasons why it's still widely used in penetration testing today.

Use Cases

Medusa is designed to test large numbers of remote systems in parallel. Its main use cases are:

- Brute-forcing user credentials across multiple services like SSH, FTP, HTTP, and more.
- Password auditing for enterprise environments.
- Testing the robustness of authentication mechanisms on multiple protocols.
- Automated and scalable login attempts across a broad range of network services.

Supported Protocols and Services

Medusa has built-in support for many services, some of the common ones include:

- SSH
- Telnet
- FTP
- POP3
- HTTP/HTTPS
- IMAP
- SMB

RDP

MySQL

VNC

And many others...

How Medusa Works

Medusa, like Hydra, performs dictionary-based and brute-force attacks. But it uses a thread-based model, making it faster and more efficient when dealing with multiple simultaneous targets. Here's a breakdown of how it works:

Connection Handling: Medusa uses a modular approach where each module corresponds to a specific service/protocol (e.g., SSH, FTP). The core of Medusa handles threading, connection setup, and management.

Threading Architecture: The tool is built to support multi-threaded operations, which means it can conduct attacks on multiple services and hosts simultaneously, maximizing performance.

Authentication Attempt: Each attempt is made using a specified username and password combination.

Response Analysis: The module analyzes the server's response and reports successful or failed attempts.

Output Management: Logs the results into a file or displays them on the console, depending on the configuration.

Basic Command Syntax

The basic command syntax for Medusa looks like this:

css

```
medusa -h [TARGET] -u [USERNAME] -P [PASSWORD LIST] -M [MODULE]
```

Here’s what each parameter means:

-h: The target IP address or hostname.

-u: Specifies a single username.

-U: Specifies a file containing a list of usernames.

-p: Specifies a single password.

-P: Points to a password list file.

-M: Specifies the module for the service (e.g., ssh, ftp).

Examples of Medusa Commands

SSH Brute Force Attack:

css

```
medusa -h 192.168.1.100 -u admin -P /path/to/password/list.txt -M ssh
```

Targets SSH service on the given IP using a single username admin and a password list.

FTP Brute Force with Username List:

css

```
medusa -h 192.168.1.100 -U /path/to/user/list.txt -p password123 -M ftp
```

Uses a list of usernames and a single password to brute-force an FTP service.

Parallel Brute Force Against Multiple Hosts:

css

```
medusa -H /path/to/hosts.txt -u root -P /path/to/password/list.txt -M ssh -t 10
```

Targets multiple hosts listed in the hosts.txt file using root as the username, a password list, and 10 threads for faster processing.

Detailed Options Explained

-H [HOST FILE]: Specifies a file containing a list of target hosts.

-t [THREADS]: Sets the number of parallel threads (default is 4).

-f: Stop after the first successful login for each host.

-T [TIMEOUT]: Sets the timeout period for each connection attempt.

-L: Verbose mode to show each attempt.

-e: Exhaustive mode, which tries additional permutations like blank passwords or reversed strings.

Advanced Features of Medusa

Speed and Performance: Medusa's ability to handle multiple threads and targets makes it extremely fast. It's ideal for testing a large number of systems at once.

Modular Design: This allows developers to add support for new services quickly. Each service has its own module, making the tool highly extensible.

Flexible Output: You can log results to different file formats, making it easier to integrate with other tools.

Comparison with Hydra

While Hydra is known for its versatility, Medusa focuses more on speed and parallel attacks. Here's a quick comparison:

Speed: Medusa is often faster due to its aggressive threading.

Scalability: Better for large-scale testing on multiple systems.

Modularity: More modular, which allows for easier customization.

User Interface: Hydra has a more verbose and interactive UI, while Medusa is more minimal.

Considerations and Limitations

Resource-Intensive: High thread count can consume significant system resources.

No CAPTCHA Handling: Medusa does not support advanced CAPTCHA mechanisms, which many modern web services use.

Dependency on Quality Wordlists: Like all brute-forcing tools, success depends on the quality of the provided username and password lists.

Medusa Cheat sheet

```
┌──(alez1㉿kali)-[/]
└─$ medusa -h
Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>

medusa: option requires an argument -- 'h'
CRITICAL: Unknown error processing command-line options.
ALERT: Host information must be supplied.

Syntax: Medusa [-h host|-H file] [-u username|-U file] [-p password|-P file] [-C file] -M module [OPT]
  -h [TEXT]    : Target hostname or IP address
  -H [FILE]    : File containing target hostnames or IP addresses
  -u [TEXT]    : Username to test
  -U [FILE]    : File containing usernames to test
  -p [TEXT]    : Password to test
  -P [FILE]    : File containing passwords to test
  -C [FILE]    : File containing combo entries. See README for more information.
  -O [FILE]    : File to append log information to
  -e [n/s/ns]  : Additional password checks ([n] No Password, [s] Password = Username)
  -M [TEXT]    : Name of the module to execute (without the .mod extension)
  -m [TEXT]    : Parameter to pass to the module. This can be passed multiple times with a
                 different parameter each time and they will all be sent to the module (i.e.
                 -m Param1 -m Param2, etc.)
  -d           : Dump all known modules
  -n [NUM]     : Use for non-default TCP port number
  -s           : Enable SSL
  -g [NUM]     : Give up after trying to connect for NUM seconds (default 3)
  -r [NUM]     : Sleep NUM seconds between retry attempts (default 3)
  -R [NUM]     : Attempt NUM retries before giving up. The total number of attempts will be NUM + 1.
  -c [NUM]     : Time to wait in usec to verify socket is available (default 500 usec).
  -t [NUM]     : Total number of logins to be tested concurrently
  -T [NUM]     : Total number of hosts to be tested concurrently
  -L           : Parallelize logins using one username per thread. The default is to process
                 the entire username before proceeding.
  -f           : Stop scanning host after first valid username/password found.
  -F           : Stop audit after first valid username/password found on any host.
  -b           : Suppress startup banner
  -q           : Display module's usage information
  -v [NUM]     : Verbose level [0 - 6 (more)]
  -w [NUM]     : Error debug level [0 - 10 (more)]
  -V           : Display version
  -Z [TEXT]    : Resume scan based on map of previous scan
```

NCRACK

1. Introduction to Ncrack

Ncrack is a high-speed network authentication cracking tool developed to help in penetration testing by identifying weak passwords on network services. Its primary focus is on brute-forcing and auditing network services such as SSH, RDP, HTTP(S), FTP, and many others. Ncrack stands out because of its speed, efficiency, and customizability, making it an essential tool in the toolkit of security professionals.

2. History and Development

Ncrack was developed by the Nmap Project team, led by Fyodor (the creator of Nmap). It was first released in twenty-ten (2010) during the Google Summer of Code project. The idea behind Ncrack was to create a tool that could rival existing password-cracking tools like Hydra and Medusa but with better efficiency and reliability. Ncrack has been crafted to support large-scale network auditing, and it's written in C language, allowing for low-level optimizations and fine-grained control.

3. Use Cases

Ncrack is particularly effective for:

Testing authentication security on critical network services such as SSH, RDP, and HTTPS.

Performing password auditing on large enterprise networks.

Evaluating the robustness of authentication mechanisms and determining how resistant they are to brute-force attacks.

Penetration testing of firewalls and VPN gateways to identify weak authentication configurations.

4. Supported Protocols and Services

Ncrack supports a wide array of protocols, including:

SSH

RDP

FTP

Telnet

HTTP(S)

POP3

VNC

SMTP

MySQL

PostgreSQL

SMB

NTP

And more...

5. How Ncrack Works

Ncrack's architecture focuses on high-speed, low-noise network authentication cracking. Here's a breakdown of its process:

Connection Pooling: Ncrack uses a connection pooling mechanism to maintain multiple concurrent connections. This allows it to manage a large number of simultaneous attempts without overwhelming the target system.

Adaptive Timing: Ncrack adapts to the target's response time, ensuring it doesn't exceed the system's capacity. This feature reduces the risk of account lockouts or triggering IDS/IPS systems.

Parallel Connection Handling: Multiple threads are used to handle different protocols simultaneously, maximizing efficiency.

Multi-Protocol Attacks: Ncrack can target multiple services on the same host or different hosts, making it ideal for broad network testing.

6. Basic Command Syntax

Ncrack's syntax is similar to Nmap's, making it easy for users familiar with Nmap. The basic command structure is:

css

ncrack [OPTIONS] <target specification>

7. Examples of Ncrack Commands

Basic SSH Brute Force Attack:

css

```
ncrack -p 22 -u root -P /path/to/passwords.txt 192.168.1.100
```

-p 22: Specifies SSH port.

-u root: Defines the username.

-P /path/to/passwords.txt: Uses a password list for brute-forcing.

Brute Forcing Multiple Services on One Host:

css

```
ncrack -p 22,80,443 192.168.1.100
```

This command targets SSH (22), HTTP (80), and HTTPS (443) on a single IP.

Brute Force Against Multiple Hosts Using an Input File:

css

```
ncrack -iL /path/to/hosts.txt -p ssh
```

-iL: Specifies an input file with a list of target hosts.

Brute Forcing RDP Service:

css

```
ncrack -p 3389 -u admin -P /path/to/passwords.txt --ssl 192.168.1.200
```

Targets RDP (port 3389) over SSL.

8. Detailed Options Explained

-p [PORTS]: Specifies target ports for various services.

-u [USERNAME]: Single username to use during the attack.

-U [USERLIST]: A file containing a list of usernames.

-P [PASSWORD LIST]: Specifies a file with a list of passwords.

--ssl: Use SSL/TLS for encrypted services like RDP or HTTPS.

-T [TEMPLATE]: Control the timing template (like in Nmap).

--pairwise: Conduct pairwise username and password attacks (useful for testing specific combinations).

--min-rate and --max-rate: Sets the minimum and maximum rate for connection attempts.

9. Advanced Features of Ncrack

Timing and Performance Optimization: Ncrack's adaptive timing can dynamically adjust attack speeds based on server response.

Fine-Grained Control: Users can define specific timing options, control connection limits, and set delays to avoid detection.

Smart Password Retry Logic: Ncrack avoids repeating passwords that are likely to trigger account lockouts.

Resumability: If an attack is interrupted, Ncrack can resume from where it left off, making it ideal for lengthy testing campaigns.

10. Inner Workings and Architecture

Ncrack's inner workings are based on:

Connection State Machines: Each protocol has its own state machine to handle authentication logic.

Pluggable Modules: Each service (e.g., SSH, RDP) has its own module, allowing for easy extension.

Connection and Timing Management: Ncrack carefully manages connection retries, timeouts, and protocol-specific behavior to optimize performance.

11. Ncrack vs. Other Tools

Compared to other tools like Hydra and Medusa, Ncrack is:

Faster for Large-Scale Testing: Ncrack's connection pooling and timing control make it better suited for large networks.

Less Resource Intensive: Ncrack's adaptive timing prevents unnecessary resource consumption.

More Reliable on Encrypted Services: Ncrack handles SSL/TLS authentication better, making it ideal for HTTPS and RDP.

12. Considerations and Limitations

Noise Generation: Ncrack can still generate significant traffic, making it detectable by IDS/IPS.

Wordlist Dependence: Like other brute-force tools, success depends on the quality of username and password lists.

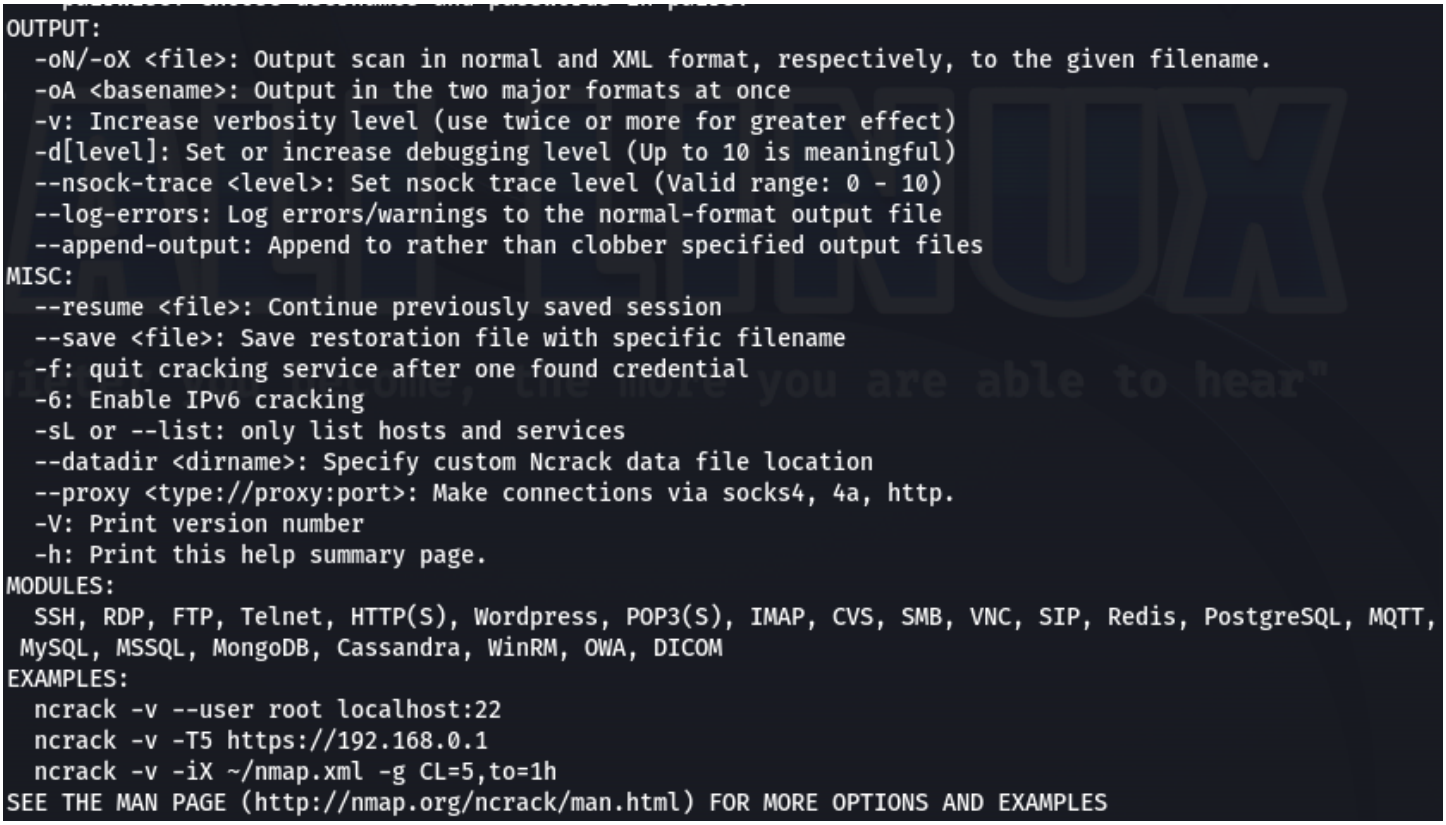
Protocol-Specific Modules: Some protocols (e.g., SMB, RDP) require custom modules, which may not always be up-to-date.

13. Conclusion

Ncrack is a robust, high-speed network authentication tool that excels in large-scale password cracking and auditing. Its adaptive timing and connection handling give it an edge over traditional tools like Hydra and Medusa, making it the go-to tool for large penetration tests. However, like all brute-forcing tools, it should be used responsibly and within legal boundaries.

14. Ncrack Cheat Sheet

```
Ncrack 0.7 ( http://ncrack.org )
Usage: ncrack [Options] {target and service specification}
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iX <inputfilename>: Input from Nmap's -oX XML output format
  -iN <inputfilename>: Input from Nmap's -oN Normal output format
  -iL <inputfilename>: Input from list of hosts/networks
  --exclude <host1[,host2][,host3],...>: Exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
SERVICE SPECIFICATION:
  Can pass target specific services in <service>://target (standard) notation or
  using -p which will be applied to all hosts in non-standard notation.
  Service arguments can be specified to be host-specific, type of service-specific
  (-m) or global (-g). Ex: ssh://10.0.0.10,at=10,cl=30 -m ssh:at=50 -g cd=3000
  Ex2: ncrack -p ssh,ftp:3500,25 10.0.0.10 scanme.nmap.org google.com:80,ssl
  -p <service-list>: services will be applied to all non-standard notation hosts
  -m <service>:<options>: options will be applied to all services of this type
  -g <options>: options will be applied to every service globally
  Misc options:
    ssl: enable SSL over this service
    path <name>: used in modules like HTTP ('=' needs escaping if used)
    db <name>: used in modules like MongoDB to specify the database
    domain <name>: used in modules like WinRM to specify the domain
TIMING AND PERFORMANCE:
  Options which take <time> are in seconds, unless you append 'ms'
  (milliseconds), 'm' (minutes), or 'h' (hours) to the value (e.g. 30m).
  Service-specific options:
    cl (min connection limit): minimum number of concurrent parallel connections
    CL (max connection limit): maximum number of concurrent parallel connections
    at (authentication tries): authentication attempts per connection
    cd (connection delay): delay <time> between each connection initiation
    cr (connection retries): caps number of service connection attempts
    to (time-out): maximum cracking <time> for service, regardless of success so far
  -T<0-5>: Set timing template (higher is faster)
  --connection-limit <number>: threshold for total concurrent connections
  --stealthy-linear: try credentials using only one connection against each specified host
    until you hit the same host again. Overrides all other timing options.
AUTHENTICATION:
  -U <filename>: username file
  -P <filename>: password file
  --user <username_list>: comma-separated username list
  --pass <password_list>: comma-separated password list
  --passwords-first: Iterate password list for each username. Default is opposite.
  --pairwise: Choose usernames and passwords in pairs.
```



4. Patator

History and Development

Patator is a multi-purpose brute-forcing tool similar to Hydra and Medusa. It was developed by Guillaume Delcourt around twenty-thirteen (2013) to address some of the limitations of other tools. It focuses on stability and providing a comprehensive framework for brute-forcing different services.

It's written in Python, which makes it platform-independent and easy to modify for different attack scenarios.

Use Cases

Patator is mainly used for testing SSH, FTP, HTTP(S), SMB, RDP, and other protocols.

It’s ideal for brute-forcing passwords, testing vulnerability to common attacks, and automating multi-step brute-force attacks.

How It Works

Patator uses a modular approach, where each module is designed to handle a specific protocol or service.

It supports multi-threading and provides detailed logging, making it easy to track and analyze brute-force attempts.

Each module can be customized with different options like rate limits, delays, or specific patterns for the username and password combinations.

Basic Command Structure

css

patator [MODULE] [OPTIONS]

For example, to brute-force SSH:

javascript

patator ssh_login host=192.168.1.100 user=root password=FILE0 0=/path/to/password_list.txt

host: Specifies the target.

user: Defines the username.

password: Takes the password list.

Supported Modules

ftp_login: Brute-force FTP service.

ssh_login: For SSH brute-forcing.

smtp_login: Test SMTP service.

smb_login: For Windows SMB shares.

rdp_login: Remote Desktop Protocol.

http_fuzz: HTTP brute-forcing and fuzzing.

Unique Features

Stability: Patator is less likely to crash or hang during long brute-force attempts.

Advanced Logging: Keeps detailed logs for successful and failed attempts.

Timeout and Rate Control: Allows fine-tuning for avoiding detection.

Patator Cheat Sheet:

```
root@kali:~# patator --help
Patator 0.9 (https://github.com/lanjelot/patator) with python-3.7.6
Usage: patator module --help

Available modules:
  + ftp_login     : Brute-force FTP
  + ssh_login     : Brute-force SSH
  + telnet_login  : Brute-force Telnet
  + smtp_login    : Brute-force SMTP
  + smtp_vrfy     : Enumerate valid users using SMTP VRFY
  + smtp_rcpt     : Enumerate valid users using SMTP RCPT TO
  + finger_lookup : Enumerate valid users using Finger
  + http_fuzz     : Brute-force HTTP
  + rdp_gateway   : Brute-force RDP Gateway
  + ajp_fuzz      : Brute-force AJP
  + pop_login     : Brute-force POP3
  + pop_passd     : Brute-force poppassd (http://netwinsite.com/poppassd/)
  + imap_login    : Brute-force IMAP4
  + ldap_login    : Brute-force LDAP
  + dcom_login    : Brute-force DCOM
  + smb_login     : Brute-force SMB
  + smb_lookupsid : Brute-force SMB SID-lookup
  + rlogin_login  : Brute-force rlogin
  + vmauthd_login : Brute-force VMware Authentication Daemon
  + mssql_login   : Brute-force MSSQL
  + oracle_login  : Brute-force Oracle
  + mysql_login   : Brute-force MySQL
  + mysql_query   : Brute-force MySQL queries
  + rdp_login     : Brute-force RDP (NLA)
  + pgsql_login   : Brute-force PostgreSQL
  + vnc_login     : Brute-force VNC
  + dns_forward   : Forward DNS lookup
```

5. Crowbar

History and Development

Crowbar is another brute-forcing tool developed primarily for SSH private key and RDP (Remote Desktop Protocol) brute-forcing.

It’s lightweight and focuses on brute-forcing authentication methods that involve password-less logins, like SSH key-based authentication.

Use Cases

Crowbar is perfect for scenarios where traditional username and password-based attacks are not possible.

It’s mainly used for SSH key-based attacks, VNC password testing, and RDP logins.

How It Works

Crowbar uses key-based authentication for brute-forcing SSH, meaning it attempts to log in using various private keys.

It also supports brute-forcing OpenVPN and VNC.

Basic Command Structure

css

```
crowbar -b [PROTOCOL] -s [TARGET] -u [USERNAME] -k [KEYFILE] -p [PORT]
```

For example, to brute-force SSH using a private key:

css

```
crowbar -b sshkey -s 192.168.1.100 -u root -k /path/to/private_key -p 22
```

-b: Backend to use (sshkey, rdp).

-s: Target IP.

-u: Username.

-k: Path to the private key file.

-p: Port.

Unique Features

SSH Key Brute-Forcing: Handles key-based authentication, which most other tools struggle with.

Supports Multiple Protocols: Including SSH, RDP, and VNC.

Simplified Command Structure: Easy to use with minimal configuration.

Crowbar Cheat Sheet

```
└─$ crowbar -h
usage: Usage: use --help for further information

Crowbar is a brute force tool which supports OpenVPN, Remote Desktop Protocol, SSH Private Keys and
VNC Keys.

positional arguments:
  options

options:
  -h, --help            show this help message and exit
  -b {openvpn,rdp,sshkey,vnckey}, --brute {openvpn,rdp,sshkey,vnckey}
                        Target service
  -s SERVER, --server SERVER
                        Static target
  -S SERVER_FILE, --serverfile SERVER_FILE
                        Multiple targets stored in a file
  -u USERNAME [USERNAME ...], --username USERNAME [USERNAME ...]
                        Static name to login with
  -U USERNAME_FILE, --usernamefile USERNAME_FILE
                        Multiple names to login with, stored in a file
  -n THREAD, --number THREAD
                        Number of threads to be active at once
  -l FILE, --log FILE   Log file (only write attempts)
  -o FILE, --output FILE
                        Output file (write everything else)
  -c PASSWD, --passwd PASSWD
                        Static password to login with
  -C FILE, --passwdfile FILE
                        Multiple passwords to login with, stored in a file
  -t TIMEOUT, --timeout TIMEOUT
                        [SSH] How long to wait for each thread (seconds)
  -p PORT, --port PORT  Alter the port if the service is not using the default value
  -k KEY_FILE, --keyfile KEY_FILE
                        [SSH/VNC] (Private) Key file or folder containing multiple files
  -m CONFIG, --config CONFIG
                        [OpenVPN] Configuration file
  -d, --discover        Port scan before attacking open ports
  -v, --verbose         Enable verbose output (-vv for more)
  -D, --debug           Enable debug mode
  -q, --quiet           Only display successful logins
```

6. THC-pptp-bruter

History and Development

Developed by the Hacker's Choice (THC) group, THC-pptp-bruter is a specialized brute-forcing tool focused on the PPTP VPN protocol.

Released around twenty-oh-four (2004), it's still used to test the security of VPN networks that use the PPTP protocol.

Use Cases

Testing and auditing PPTP VPN connections.

Identifying weak passwords in VPN authentication.

Evaluating the robustness of VPN configurations.

How It Works

THC-pptp-bruter uses multi-threading to brute-force the CHAP (Challenge-Handshake Authentication Protocol) used in PPTP.

It's designed to be high-speed and efficient, making it perfect for large-scale VPN testing.

Basic Command Structure

css

```
thc-pptp-bruter -u [USERNAME] -p [PASSWORD] [TARGET]
```

For example:

css

thc-pptp-bruter -u admin -p password123 192.168.1.1

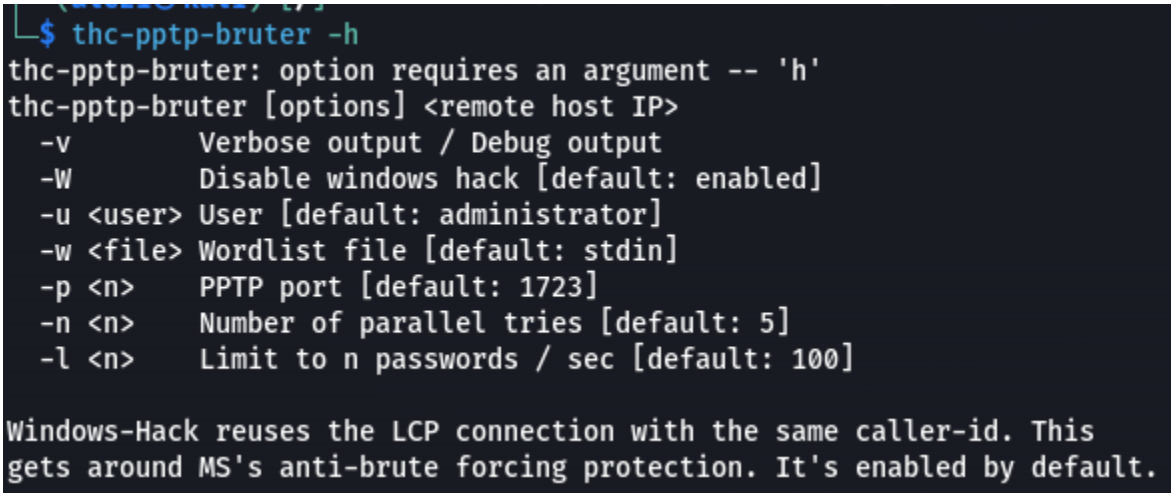
Unique Features

High-Speed Brute-Forcing: Optimized for VPN testing.

Lightweight: Minimal dependencies and small footprint.

Focus on VPN: One of the few tools designed for PPTP VPN testing.

Thc-pptp-Bruter Cheat sheet



7. BruteSpray

History and Development

BruteSpray is a relatively newer tool designed as a wrapper around Nmap. It automates the process of using Ncrack, Hydra, or Medusa after discovering open ports using Nmap.

Developed to simplify and streamline brute-forcing after initial network scanning.

Use Cases

Automatically brute-forcing discovered services on multiple hosts.

Ideal for large-scale network pentests.

Integrates easily with other tools in the Kali Linux arsenal.

How It Works

BruteSpray parses Nmap scan results and uses the output to identify services to brute-force.

It then invokes Ncrack, Hydra, or Medusa based on the configuration.

Basic Command Structure

css

```
brutespray --file [NMAP_RESULT_FILE] -U [USERLIST] -P [PASSWORDLIST]
```

For example:

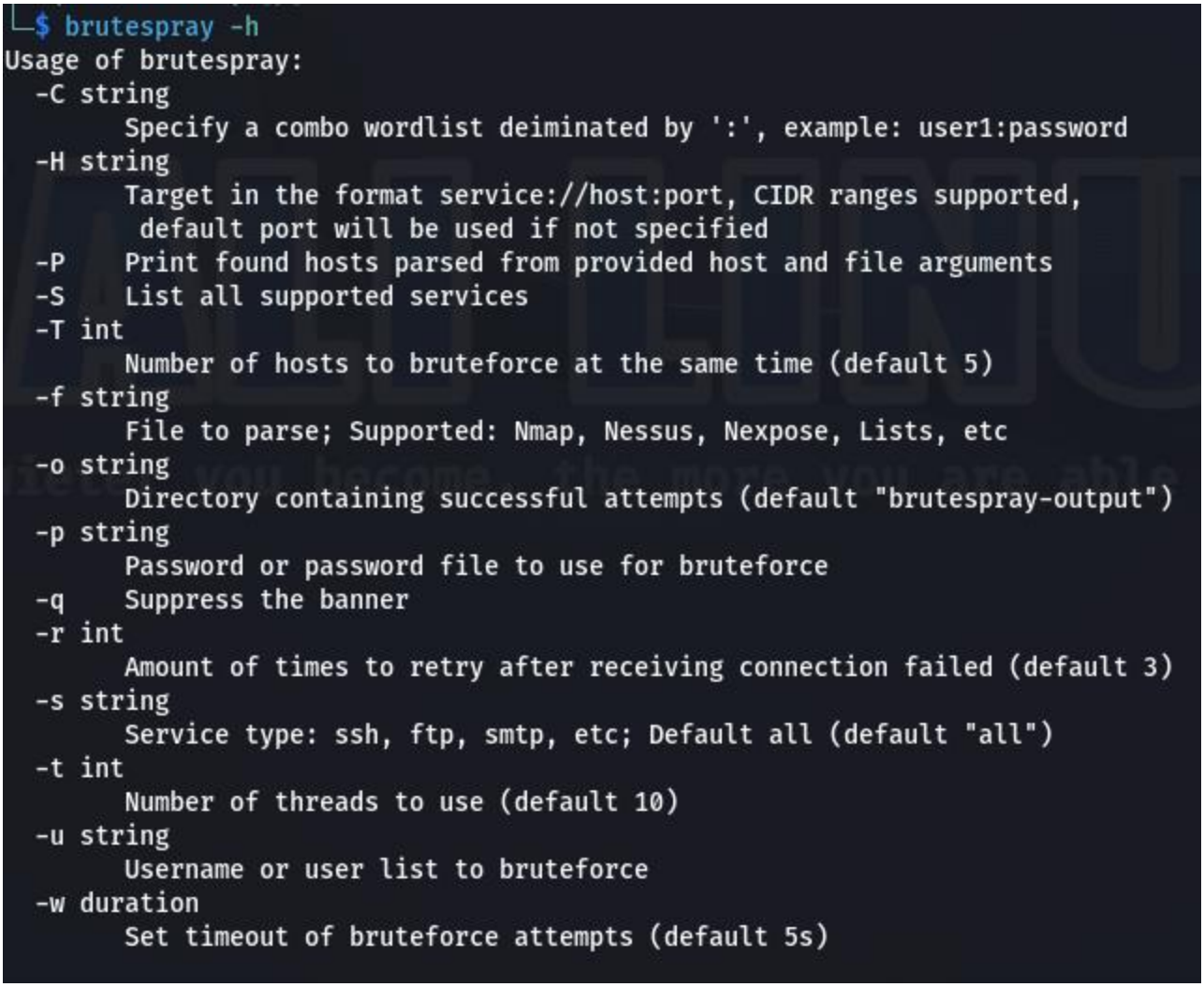
css

```
brutespray --file nmap_scan.xml -U users.txt -P passwords.txt
```

Unique Features

- Automatic Integration with Nmap: Automates post-scan brute-forcing.
- Supports Multiple Tools: Can use Ncrack, Hydra, or Medusa based on availability.
- Simplified Configuration: Minimal setup required for large-scale testing.

Brutespray Cheat Sheet



8. HASHCAT

HashCat is a powerful password recovery tool that specializes in cracking hashed passwords using CPU and GPU acceleration. Here's a detailed overview of HashCat for you, along with examples to get started:

History and Development

HashCat was initially created in the early 2000s by Jens "atom" Steube. It started as a CPU-only tool named "oclHashcat," but it evolved to include GPU support, making it much faster for large-scale password cracking. Over time, it became a unified version supporting both CPU and GPU, and it is now known simply as HashCat. It's open-source and has a strong community for updates and support, making it popular among security professionals and penetration testers.

Use Cases

Password Recovery: Retrieving lost passwords from encrypted files, databases, or systems.

Pentesting: Testing the strength of password policies during penetration tests.

Digital Forensics: Extracting credentials from captured data in forensic investigations.

Red Teaming: Simulating attacks to determine the resilience of a system's password security.

When to Use HashCat

HashCat is ideal for situations where you need to crack hashes quickly. It supports a wide range of hashing algorithms such as MD5, SHA1, SHA256, bcrypt, NTLM, and many others. You should use HashCat when:

The target is a hashed password and needs to be reversed.

Performing security audits or pentests to evaluate password security.

Handling large hash dumps and want GPU-accelerated processing.

Features of HashCat

Multi-Device Support: Works with CPUs, GPUs, and other hardware accelerators like FPGAs.

Algorithm Support: Supports over two hundred algorithms, including NTLM, MD5, SHA family, bcrypt, and more.

Rule-Based Attacks: Allows creating rules to modify inputs and test various password combinations.

Mask Attack: Uses pattern matching for faster cracking (e.g., targeting specific character sets or lengths).

Hybrid Attack: Combines dictionary and mask attacks to broaden search space.

Support for Distributed Cracking: Can split the workload across multiple machines.

Session Management: Easily pause and resume cracking sessions.

Tuning and Optimization: Advanced users can customize configurations for optimized performance.

HashCat in Kali Linux

HashCat is pre-installed in Kali Linux. You can find it using the terminal command which hashcat or directly in the /usr/bin/ directory. If not installed, you can add it using the command:

bash

sudo apt-get install hashcat

Conditions to Use HashCat

Access to Hashes: You need the hashed version of the password.

Supported Hardware: For GPU acceleration, you need a compatible GPU and the necessary drivers installed (NVIDIA or AMD).

Dictionary or Wordlist: A wordlist like rockyou.txt is commonly used for dictionary attacks.

Good System Resources: For GPU-accelerated cracking, having a robust system with adequate cooling is recommended.

Basic Command Structure and Examples

Check HashCat Version and Help

bash

hashcat -I

hashcat --help

This displays the version, available devices, and help documentation.

Basic Dictionary Attack Let’s say you have a hash file (hash.txt) containing the hash of a password and want to crack it using a dictionary (wordlist.txt):

bash

```
hashcat -m 0 -a 0 hash.txt wordlist.txt
```

- -m 0 specifies the hash type (e.g., MD5).
- -a 0 is for a dictionary attack.

Mask Attack Example If you know the password format, use a mask:

bash

```
hashcat -m 1000 -a 3 hash.txt ?u?u?u?u?d?d?d?d
```

This command cracks an NTLM hash (-m 1000) using a pattern where the password is expected to have four uppercase letters and four digits.

Brute-Force Attack If you have no idea about the password format, you can use a brute-force attack (though it’s slow):

bash

```
hashcat -m 0 -a 3 hash.txt ?a?a?a?a?a?a
```

- ?a means all possible characters (lowercase, uppercase, digits, and special symbols).

Rule-Based Attack HashCat can use rules to modify inputs dynamically:

bash

```
hashcat -m 0 -a 0 -r rules/best64.rule hash.txt wordlist.txt
```

This applies the best64.rule to the wordlist, generating modified versions of each word (e.g., adding numbers or changing case).

Common HashCat Attack Modes

Dictionary Attack (-a 0): Tests against a wordlist.

Combinator Attack (-a 1): Combines words from two dictionaries.

Mask Attack (-a 3): Tries password patterns (e.g., uppercase + digits).

Hybrid Attack (-a 6/7): Combines dictionary and mask attacks.

Examples of Hash Formats

MD5 Hash: -m 0

SHA1 Hash: -m 100

bcrypt: -m 3200

NTLM: -m 1000 You can get the complete list of hash formats using:

bash

```
hashcat --help | grep Hash-Types
```

Advanced Use Cases

Cracking WPA2 Wi-Fi Passwords If you capture a WPA2 handshake (handshake.cap), convert it to .hccapx format and run:

bash

```
hashcat -m 2500 -a 0 handshake.hccapx wordlist.txt
```

This cracks the Wi-Fi password using the captured handshake.

Cracking Encrypted Archives For cracking zip file passwords, extract the hash and run:

bash

```
hashcat -m 13600 hash.txt wordlist.txt
```

Tips for Effective Cracking

Use GPU Acceleration: For faster results, ensure GPU drivers are properly installed.

Session Management: Use --session to save and resume sessions.

Optimize Rules: Choose rules based on password patterns to avoid unnecessary combinations.

HashCAT Cheat Sheet:

hashcat (v6.2.6) starting in help mode

Usage: hashcat [options]... hash|hashfile|hccapxfile [dictionary|mask|directory]...

- [Options] -

Options Short / Long | Type | Description | Example

```
==============================+======+=========================================
==========+======================
-m, --hash-type            | Num  | Hash-type, references below (otherwise autodetect)  | -m 1000
-a, --attack-mode          | Num  | Attack-mode, see references below                   | -a 3
-V, --version              |      | Print version                                       |
-h, --help                 |      | Print help                                          |
    --quiet                |      | Suppress output                                     |
    --hex-charset          |      | Assume charset is given in hex                      |
    --hex-salt             |      | Assume salt is given in hex                         |
    --hex-wordlist         |      | Assume words in wordlist are given in hex           |
    --force                |      | Ignore warnings                                     |
    --deprecated-check-disable |  | Enable deprecated plugins                           |
    --status               |      | Enable automatic update of the status screen        |
    --status-json          |      | Enable JSON format for status output                |
    --status-timer         | Num  | Sets seconds between status screen updates to X     | --status-timer=1
    --stdin-timeout-abort  | Num  | Abort if there is no input from stdin for X seconds | --stdin-timeout-
abort=300
    --machine-readable     |      | Display the status view in a machine-readable format |
    --keep-guessing        |      | Keep guessing the hash after it has been cracked    |
    --self-test-disable    |      | Disable self-test functionality on startup          |
    --loopback             |      | Add new plains to induct directory                  |
    --markov-hcstat2       | File | Specify hcstat2 file to use                         | --markov-hcstat2=my.hcstat2
    --markov-disable       |      | Disables markov-chains, emulates classic brute-force |
    --markov-classic       |      | Enables classic markov-chains, no per-position      |
    --markov-inverse       |      | Enables inverse markov-chains, no per-position      |
-t, --markov-threshold     | Num  | Threshold X when to stop accepting new markov-chains | -t 50
    --runtime              | Num  | Abort session after X seconds of runtime            | --runtime=10
    --session              | Str  | Define specific session name                        | --session=mysession
    --restore              |      | Restore session from --session                      |
```

--restore-disable | | Do not write restore file |

--restore-file-path | File | Specific path to restore file | --restore-file-path=x.restore

-o, --outfile | File | Define outfile for recovered hash | -o outfile.txt

--outfile-format | Str | Outfile format to use, separated with commas | --outfile-format=1,3

--outfile-autohex-disable | | Disable the use of $HEX[] in output plains |

--outfile-check-timer | Num | Sets seconds between outfile checks to X | --outfile-check-timer=30

--wordlist-autohex-disable | | Disable the conversion of $HEX[] from the wordlist |

-p, --separator | Char | Separator char for hashlists and outfile | -p :

--stdout | | Do not crack a hash, instead print candidates only |

--show | | Compare hashlist with potfile; show cracked hashes |

--left | | Compare hashlist with potfile; show uncracked hashes |

--username | | Enable ignoring of usernames in hashfile |

--remove | | Enable removal of hashes once they are cracked |

--remove-timer | Num | Update input hash file each X seconds | --remove-timer=30

--potfile-disable | | Do not write potfile |

--potfile-path | File | Specific path to potfile | --potfile-path=my.pot

--encoding-from | Code | Force internal wordlist encoding from X | --encoding-from=iso-8859-15

--encoding-to | Code | Force internal wordlist encoding to X | --encoding-to=utf-32le

--debug-mode | Num | Defines the debug mode (hybrid only by using rules) | --debug-mode=4

--debug-file | File | Output file for debugging rules | --debug-file=good.log

--induction-dir | Dir | Specify the induction directory to use for loopback | --induction=inducts

--outfile-check-dir | Dir | Specify the outfile directory to monitor for plains | --outfile-check-dir=x

--logfile-disable | | Disable the logfile |

--hccapx-message-pair | Num | Load only message pairs from hccapx matching X | --hccapx-message-pair=2

--nonce-error-corrections | Num | The BF size range to replace AP's nonce last bytes | --nonce-error-corrections=16

--keyboard-layout-mapping | File | Keyboard layout mapping table for special hash-modes | --keyb=german.hckmap

--truecrypt-keyfiles | File | Keyfiles to use, separated with commas | --truecrypt-keyf=x.png

--veracrypt-keyfiles | File | Keyfiles to use, separated with commas | --veracrypt-keyf=x.txt

--veracrypt-pim-start | Num | VeraCrypt personal iterations multiplier start | --veracrypt-pim-start=450

--veracrypt-pim-stop | Num | VeraCrypt personal iterations multiplier stop | --veracrypt-pim-stop=500

-b, --benchmark | | Run benchmark of selected hash-modes |

--benchmark-all | | Run benchmark of all hash-modes (requires -b) |

--speed-only | | Return expected speed of the attack, then quit |

--progress-only | | Return ideal progress step size and time to process |

-c, --segment-size | Num | Sets size in MB to cache from the wordfile to X | -c 32

--bitmap-min | Num | Sets minimum bits allowed for bitmaps to X | --bitmap-min=24

--bitmap-max | Num | Sets maximum bits allowed for bitmaps to X | --bitmap-max=24

--cpu-affinity | Str | Locks to CPU devices, separated with commas | --cpu-affinity=1,2,3

--hook-threads | Num | Sets number of threads for a hook (per compute unit) | --hook-threads=8

--hash-info | | Show information for each hash-mode |

--example-hashes | | Alias of --hash-info |

--backend-ignore-cuda | | Do not try to open CUDA interface on startup |

--backend-ignore-hip | | Do not try to open HIP interface on startup |

--backend-ignore-metal | | Do not try to open Metal interface on startup |

--backend-ignore-opencl | | Do not try to open OpenCL interface on startup |

-I, --backend-info | | Show system/evironment/backend API info | -I or -II

-d, --backend-devices | Str | Backend devices to use, separated with commas | -d 1

-D, --opencl-device-types | Str | OpenCL device-types to use, separated with commas | -D 1

-O, --optimized-kernel-enable | | Enable optimized kernels (limits password length) |

-M, --multiply-accel-disable | | Disable multiply kernel-accel with processor count |

-w, --workload-profile | Num | Enable a specific workload profile, see pool below | -w 3

-n, --kernel-accel | Num | Manual workload tuning, set outerloop step size to X | -n 64

-u, --kernel-loops | Num | Manual workload tuning, set innerloop step size to X | -u 256

-T, --kernel-threads | Num | Manual workload tuning, set thread count to X | -T 64

--backend-vector-width | Num | Manually override backend vector-width to X | --backend-vector=4

--spin-damp | Num | Use CPU for device synchronization, in percent | --spin-damp=10

--hwmon-disable | | Disable temperature and fanspeed reads and triggers |

--hwmon-temp-abort | Num | Abort if temperature reaches X degrees Celsius | --hwmon-temp-abort=100

--scrypt-tmto | Num | Manually override TMTO value for scrypt to X | --scrypt-tmto=3

-s, --skip | Num | Skip X words from the start | -s 1000000

-l, --limit | Num | Limit X words from the start + skipped words | -l 1000000

--keyspace | | Show keyspace base:mod values and quit |

-j, --rule-left | Rule | Single rule applied to each word from left wordlist | -j 'c'

-k, --rule-right | Rule | Single rule applied to each word from right wordlist | -k '^-'

-r, --rules-file | File | Multiple rules applied to each word from wordlists | -r rules/best64.rule

-g, --generate-rules | Num | Generate X random rules | -g 10000

--generate-rules-func-min | Num | Force min X functions per rule |

--generate-rules-func-max | Num | Force max X functions per rule |

--generate-rules-func-sel | Str | Pool of rule operators valid for random rule engine | --generate-rules-func-sel=ioTlc

--generate-rules-seed | Num | Force RNG seed set to X |

-1, --custom-charset1 | CS | User-defined charset ?1 | -1 ?l?d?u

-2, --custom-charset2 | CS | User-defined charset ?2 | -2 ?l?d?s

-3, --custom-charset3 | CS | User-defined charset ?3 |

-4, --custom-charset4 | CS | User-defined charset ?4 |

--identify | | Shows all supported algorithms for input hashes | --identify my.hash

-i, --increment | | Enable mask increment mode |

--increment-min | Num | Start mask incrementing at X | --increment-min=4

--increment-max | Num | Stop mask incrementing at X | --increment-max=8

-S, --slow-candidates | | Enable slower (but advanced) candidate generators |

--brain-server | | Enable brain server |

--brain-server-timer | Num | Update the brain server dump each X seconds (min:60) | --brain-server-timer=300

-z, --brain-client | | Enable brain client, activates -S |

--brain-client-features | Num | Define brain client features, see below | --brain-client-features=3

--brain-host | Str | Brain server host (IP or domain) | --brain-host=127.0.0.1

--brain-port | Port | Brain server port | --brain-port=13743

--brain-password | Str | Brain server authentication password | --brain-password=bZfhCvGUSjRq

--brain-session | Hex | Overrides automatically calculated brain session | --brain-session=0x2ae611db

--brain-session-whitelist | Hex | Allow given sessions only, separated with commas | --brain-session-whitelist=0x2ae611db

- [Hash modes] -

#	Name	Category
900	MD4	Raw Hash
0	MD5	Raw Hash
100	SHA1	Raw Hash
1300	SHA2-224	Raw Hash
1400	SHA2-256	Raw Hash
10800	SHA2-384	Raw Hash
1700	SHA2-512	Raw Hash
17300	SHA3-224	Raw Hash
17400	SHA3-256	Raw Hash
17500	SHA3-384	Raw Hash

17600 | SHA3-512 | Raw Hash

6000 | RIPEMD-160 | Raw Hash

600 | BLAKE2b-512 | Raw Hash

11700 | GOST R 34.11-2012 (Streebog) 256-bit, big-endian | Raw Hash

11800 | GOST R 34.11-2012 (Streebog) 512-bit, big-endian | Raw Hash

6900 | GOST R 34.11-94 | Raw Hash

17010 | GPG (AES-128/AES-256 (SHA-1($pass))) | Raw Hash

5100 | Half MD5 | Raw Hash

17700 | Keccak-224 | Raw Hash

17800 | Keccak-256 | Raw Hash

17900 | Keccak-384 | Raw Hash

18000 | Keccak-512 | Raw Hash

6100 | Whirlpool | Raw Hash

10100 | SipHash | Raw Hash

70 | md5(utf16le($pass)) | Raw Hash

170 | sha1(utf16le($pass)) | Raw Hash

1470 | sha256(utf16le($pass)) | Raw Hash

10870 | sha384(utf16le($pass)) | Raw Hash

1770 | sha512(utf16le($pass)) | Raw Hash

610 | BLAKE2b-512($pass.$salt) | Raw Hash salted and/or iterated

620 | BLAKE2b-512($salt.$pass) | Raw Hash salted and/or iterated

10 | md5($pass.$salt) | Raw Hash salted and/or iterated

20 | md5($salt.$pass) | Raw Hash salted and/or iterated

3800 | md5($salt.$pass.$salt) | Raw Hash salted and/or iterated

3710 | md5($salt.md5($pass)) | Raw Hash salted and/or iterated

4110 | md5($salt.md5($pass.$salt)) | Raw Hash salted and/or iterated

4010 | md5($salt.md5($salt.$pass)) | Raw Hash salted and/or iterated

21300 | md5($salt.sha1($salt.$pass)) | Raw Hash salted and/or iterated

40 | md5($salt.utf16le($pass)) | Raw Hash salted and/or iterated

2600 | md5(md5($pass)) | Raw Hash salted and/or iterated

3910 | md5(md5($pass).md5($salt)) | Raw Hash salted and/or iterated

3500 | md5(md5(md5($pass))) | Raw Hash salted and/or iterated

4400 | md5(sha1($pass)) | Raw Hash salted and/or iterated

4410 | md5(sha1($pass).$salt) | Raw Hash salted and/or iterated

20900 | md5(sha1($pass).md5($pass).sha1($pass)) | Raw Hash salted and/or iterated

21200 | md5(sha1($salt).md5($pass)) | Raw Hash salted and/or iterated

4300 | md5(strtoupper(md5($pass))) | Raw Hash salted and/or iterated

30 | md5(utf16le($pass).$salt) | Raw Hash salted and/or iterated

110 | sha1($pass.$salt) | Raw Hash salted and/or iterated

120 | sha1($salt.$pass) | Raw Hash salted and/or iterated

4900 | sha1($salt.$pass.$salt) | Raw Hash salted and/or iterated

4520 | sha1($salt.sha1($pass)) | Raw Hash salted and/or iterated

24300 | sha1($salt.sha1($pass.$salt)) | Raw Hash salted and/or iterated

140 | sha1($salt.utf16le($pass)) | Raw Hash salted and/or iterated

19300 | sha1($salt1.$pass.$salt2) | Raw Hash salted and/or iterated

14400 | sha1(CX) | Raw Hash salted and/or iterated

4700 | sha1(md5($pass)) | Raw Hash salted and/or iterated

4710 | sha1(md5($pass).$salt) | Raw Hash salted and/or iterated

21100 | sha1(md5($pass.$salt)) | Raw Hash salted and/or iterated

18500 | sha1(md5(md5($pass))) | Raw Hash salted and/or iterated

4500 | sha1(sha1($pass)) | Raw Hash salted and/or iterated

4510 | sha1(sha1($pass).$salt) | Raw Hash salted and/or iterated

5000 | sha1(sha1($salt.$pass.$salt)) | Raw Hash salted and/or iterated

130 | sha1(utf16le($pass).$salt) | Raw Hash salted and/or iterated

1410 | sha256($pass.$salt) | Raw Hash salted and/or iterated

1420 | sha256($salt.$pass) | Raw Hash salted and/or iterated

22300 | sha256($salt.$pass.$salt) | Raw Hash salted and/or iterated

20720 | sha256($salt.sha256($pass)) | Raw Hash salted and/or iterated

21420 | sha256($salt.sha256_bin($pass)) | Raw Hash salted and/or iterated

1440 | sha256($salt.utf16le($pass)) | Raw Hash salted and/or iterated

20800 | sha256(md5($pass)) | Raw Hash salted and/or iterated

20710 | sha256(sha256($pass).$salt) | Raw Hash salted and/or iterated

21400 | sha256(sha256_bin($pass)) | Raw Hash salted and/or iterated

1430 | sha256(utf16le($pass).$salt) | Raw Hash salted and/or iterated

10810 | sha384($pass.$salt) | Raw Hash salted and/or iterated

10820 | sha384($salt.$pass) | Raw Hash salted and/or iterated

10840 | sha384($salt.utf16le($pass)) | Raw Hash salted and/or iterated

10830 | sha384(utf16le($pass).$salt) | Raw Hash salted and/or iterated

1710 | sha512($pass.$salt) | Raw Hash salted and/or iterated

1720 | sha512($salt.$pass) | Raw Hash salted and/or iterated

1740 | sha512($salt.utf16le($pass)) | Raw Hash salted and/or iterated

1730 | sha512(utf16le($pass).$salt) | Raw Hash salted and/or iterated

50 | HMAC-MD5 (key = $pass) | Raw Hash authenticated

60 | HMAC-MD5 (key = $salt) | Raw Hash authenticated

150 | HMAC-SHA1 (key = $pass) | Raw Hash authenticated

160 | HMAC-SHA1 (key = $salt) | Raw Hash authenticated

1450 | HMAC-SHA256 (key = $pass) | Raw Hash authenticated

1460 | HMAC-SHA256 (key = $salt) | Raw Hash authenticated

1750 | HMAC-SHA512 (key = $pass) | Raw Hash authenticated

1760 | HMAC-SHA512 (key = $salt) | Raw Hash authenticated

11750 | HMAC-Streebog-256 (key = $pass), big-endian | Raw Hash authenticated

11760 | HMAC-Streebog-256 (key = $salt), big-endian | Raw Hash authenticated

11850 | HMAC-Streebog-512 (key = $pass), big-endian | Raw Hash authenticated

11860 | HMAC-Streebog-512 (key = $salt), big-endian | Raw Hash authenticated

28700 | Amazon AWS4-HMAC-SHA256 | Raw Hash authenticated

11500 | CRC32 | Raw Checksum

27900 | CRC32C | Raw Checksum

28000 | CRC64Jones | Raw Checksum

18700 | Java Object hashCode() | Raw Checksum

25700 | MurmurHash | Raw Checksum

27800 | MurmurHash3 | Raw Checksum

14100 | 3DES (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

14000 | DES (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

26401 | AES-128-ECB NOKDF (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

26402 | AES-192-ECB NOKDF (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

26403 | AES-256-ECB NOKDF (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

15400 | ChaCha20 | Raw Cipher, Known-plaintext attack

14500 | Linux Kernel Crypto API (2.4) | Raw Cipher, Known-plaintext attack

14900 | Skip32 (PT = $salt, key = $pass) | Raw Cipher, Known-plaintext attack

11900 | PBKDF2-HMAC-MD5 | Generic KDF

12000 | PBKDF2-HMAC-SHA1 | Generic KDF

10900 | PBKDF2-HMAC-SHA256 | Generic KDF

12100 | PBKDF2-HMAC-SHA512 | Generic KDF

8900 | scrypt | Generic KDF

400 | phpass | Generic KDF

16100 | TACACS+ | Network Protocol

11400 | SIP digest authentication (MD5) | Network Protocol

5300 | IKE-PSK MD5 | Network Protocol

5400 | IKE-PSK SHA1 | Network Protocol

25100 | SNMPv3 HMAC-MD5-96 | Network Protocol

25000 | SNMPv3 HMAC-MD5-96/HMAC-SHA1-96 | Network Protocol

25200 | SNMPv3 HMAC-SHA1-96 | Network Protocol

26700 | SNMPv3 HMAC-SHA224-128 | Network Protocol

26800 | SNMPv3 HMAC-SHA256-192 | Network Protocol

26900 | SNMPv3 HMAC-SHA384-256 | Network Protocol

27300 | SNMPv3 HMAC-SHA512-384 | Network Protocol

2500 | WPA-EAPOL-PBKDF2 | Network Protocol

2501 | WPA-EAPOL-PMK | Network Protocol

22000 | WPA-PBKDF2-PMKID+EAPOL | Network Protocol

22001 | WPA-PMK-PMKID+EAPOL | Network Protocol

16800 | WPA-PMKID-PBKDF2 | Network Protocol

16801 | WPA-PMKID-PMK | Network Protocol

7300 | IPMI2 RAKP HMAC-SHA1 | Network Protocol

10200 | CRAM-MD5 | Network Protocol

16500 | JWT (JSON Web Token) | Network Protocol

29200 | Radmin3 | Network Protocol

19600 | Kerberos 5, etype 17, TGS-REP | Network Protocol

19800 | Kerberos 5, etype 17, Pre-Auth | Network Protocol

28800 | Kerberos 5, etype 17, DB | Network Protocol

19700 | Kerberos 5, etype 18, TGS-REP | Network Protocol

19900 | Kerberos 5, etype 18, Pre-Auth | Network Protocol

28900 | Kerberos 5, etype 18, DB | Network Protocol

7500 | Kerberos 5, etype 23, AS-REQ Pre-Auth | Network Protocol

13100 | Kerberos 5, etype 23, TGS-REP | Network Protocol

18200 | Kerberos 5, etype 23, AS-REP | Network Protocol

5500 | NetNTLMv1 / NetNTLMv1+ESS | Network Protocol

27000 | NetNTLMv1 / NetNTLMv1+ESS (NT) | Network Protocol

5600 | NetNTLMv2 | Network Protocol

27100 | NetNTLMv2 (NT) | Network Protocol

29100 | Flask Session Cookie ($salt.$salt.$pass) | Network Protocol

4800 | iSCSI CHAP authentication, MD5(CHAP) | Network Protocol

8500 | RACF | Operating System

6300 | AIX {smd5} | Operating System

6700 | AIX {ssha1} | Operating System

6400 | AIX {ssha256} | Operating System

6500 | AIX {ssha512} | Operating System

3000 | LM | Operating System

19000 | QNX /etc/shadow (MD5) | Operating System

19100 | QNX /etc/shadow (SHA256) | Operating System

19200 | QNX /etc/shadow (SHA512) | Operating System

15300 | DPAPI masterkey file v1 (context 1 and 2) | Operating System

15310 | DPAPI masterkey file v1 (context 3) | Operating System

15900 | DPAPI masterkey file v2 (context 1 and 2) | Operating System

15910 | DPAPI masterkey file v2 (context 3) | Operating System

7200 | GRUB 2 | Operating System

12800 | MS-AzureSync PBKDF2-HMAC-SHA256 | Operating System

12400 | BSDi Crypt, Extended DES | Operating System

1000 | NTLM | Operating System

9900 | Radmin2 | Operating System

5800 | Samsung Android Password/PIN | Operating System

28100 | Windows Hello PIN/Password | Operating System

13800 | Windows Phone 8+ PIN/password | Operating System

2410 | Cisco-ASA MD5 | Operating System

9200 | Cisco-IOS 8 (PBKDF2-SHA256) | Operating System

9300 | Cisco-IOS 9 (scrypt) | Operating System

5700 | Cisco-IOS type 4 (SHA256) | Operating System

2400 | Cisco-PIX MD5 | Operating System

8100 | Citrix NetScaler (SHA1) | Operating System

22200 | Citrix NetScaler (SHA512) | Operating System

1100 | Domain Cached Credentials (DCC), MS Cache | Operating System

2100 | Domain Cached Credentials 2 (DCC2), MS Cache 2 | Operating System

7000 | FortiGate (FortiOS) | Operating System

26300 | FortiGate256 (FortiOS256) | Operating System

125 | ArubaOS | Operating System

501 | Juniper IVE | Operating System

22 | Juniper NetScreen/SSG (ScreenOS) | Operating System

15100 | Juniper/NetBSD sha1crypt | Operating System

26500 | iPhone passcode (UID key + System Keybag) | Operating System

122 | macOS v10.4, macOS v10.5, macOS v10.6 | Operating System

1722 | macOS v10.7 | Operating System

7100 | macOS v10.8+ (PBKDF2-SHA512) | Operating System

3200 | bcrypt $2*$, Blowfish (Unix) | Operating System

500 | md5crypt, MD5 (Unix), Cisco-IOS 1 (MD5) | Operating System

1500 | descrypt, DES (Unix), Traditional DES | Operating System

29000 | sha1($salt.sha1(utf16le($username).':'.utf16le($pass))) | Operating System

7400 | sha256crypt 5, SHA256 (Unix) | Operating System

1800 | sha512crypt 6, SHA512 (Unix) | Operating System

24600 | SQLCipher | Database Server

131 | MSSQL (2000) | Database Server

132 | MSSQL (2005) | Database Server

1731 | MSSQL (2012, 2014) | Database Server

24100 | MongoDB ServerKey SCRAM-SHA-1 | Database Server

24200 | MongoDB ServerKey SCRAM-SHA-256 | Database Server

12 | PostgreSQL | Database Server

11100 | PostgreSQL CRAM (MD5) | Database Server

28600 | PostgreSQL SCRAM-SHA-256 | Database Server

3100 | Oracle H: Type (Oracle 7+) | Database Server

112 | Oracle S: Type (Oracle 11+) | Database Server

12300 | Oracle T: Type (Oracle 12+) | Database Server

7401 | MySQL A (sha256crypt) | Database Server

11200 | MySQL CRAM (SHA1) | Database Server

200 | MySQL323 | Database Server

300 | MySQL4.1/MySQL5 | Database Server

8000 | Sybase ASE | Database Server

8300 | DNSSEC (NSEC3) | FTP, HTTP, SMTP, LDAP Server

25900 | KNX IP Secure - Device Authentication Code | FTP, HTTP, SMTP, LDAP Server

16400 | CRAM-MD5 Dovecot | FTP, HTTP, SMTP, LDAP Server

1411 | SSHA-256(Base64), LDAP {SSHA256} | FTP, HTTP, SMTP, LDAP Server

1711 | SSHA-512(Base64), LDAP {SSHA512} | FTP, HTTP, SMTP, LDAP Server

24900 | Dahua Authentication MD5 | FTP, HTTP, SMTP, LDAP Server

10901 | RedHat 389-DS LDAP (PBKDF2-HMAC-SHA256) | FTP, HTTP, SMTP, LDAP Server

15000 | FileZilla Server >= 0.9.55 | FTP, HTTP, SMTP, LDAP Server

12600 | ColdFusion 10+ | FTP, HTTP, SMTP, LDAP Server

1600 | Apache $apr1$ MD5, md5apr1, MD5 (APR) | FTP, HTTP, SMTP, LDAP Server

141 | Episerver 6.x < .NET 4 | FTP, HTTP, SMTP, LDAP Server

1441 | Episerver 6.x >= .NET 4 | FTP, HTTP, SMTP, LDAP Server

1421 | hMailServer | FTP, HTTP, SMTP, LDAP Server

101 | nsldap, SHA-1(Base64), Netscape LDAP SHA | FTP, HTTP, SMTP, LDAP Server

111 | nsldaps, SSHA-1(Base64), Netscape LDAP SSHA | FTP, HTTP, SMTP, LDAP Server

7700 | SAP CODVN B (BCODE) | Enterprise Application Software (EAS)

7701 | SAP CODVN B (BCODE) from RFC_READ_TABLE | Enterprise Application Software (EAS)

7800 | SAP CODVN F/G (PASSCODE) | Enterprise Application Software (EAS)

7801 | SAP CODVN F/G (PASSCODE) from RFC_READ_TABLE | Enterprise Application Software (EAS)

10300 | SAP CODVN H (PWDSALTEDHASH) iSSHA-1 | Enterprise Application Software (EAS)

133 | PeopleSoft | Enterprise Application Software (EAS)

13500 | PeopleSoft PS_TOKEN | Enterprise Application Software (EAS)

21500 | SolarWinds Orion | Enterprise Application Software (EAS)

21501 | SolarWinds Orion v2 | Enterprise Application Software (EAS)

24 | SolarWinds Serv-U | Enterprise Application Software (EAS)

8600 | Lotus Notes/Domino 5 | Enterprise Application Software (EAS)

8700 | Lotus Notes/Domino 6 | Enterprise Application Software (EAS)

9100 | Lotus Notes/Domino 8 | Enterprise Application Software (EAS)

26200 | OpenEdge Progress Encode | Enterprise Application Software (EAS)

20600 | Oracle Transportation Management (SHA256) | Enterprise Application Software (EAS)

4711 | Huawei sha1(md5($pass).$salt) | Enterprise Application Software (EAS)

20711 | AuthMe sha256 | Enterprise Application Software (EAS)

22400 | AES Crypt (SHA256) | Full-Disk Encryption (FDE)

27400 | VMware VMX (PBKDF2-HMAC-SHA1 + AES-256-CBC) | Full-Disk Encryption (FDE)

14600 | LUKS v1 (legacy) | Full-Disk Encryption (FDE)

29541 | LUKS v1 RIPEMD-160 + AES | Full-Disk Encryption (FDE)

29542 | LUKS v1 RIPEMD-160 + Serpent | Full-Disk Encryption (FDE)

29543 | LUKS v1 RIPEMD-160 + Twofish | Full-Disk Encryption (FDE)

29511 | LUKS v1 SHA-1 + AES | Full-Disk Encryption (FDE)

29512 | LUKS v1 SHA-1 + Serpent | Full-Disk Encryption (FDE)

29513 | LUKS v1 SHA-1 + Twofish | Full-Disk Encryption (FDE)

29521 | LUKS v1 SHA-256 + AES | Full-Disk Encryption (FDE)

29522 | LUKS v1 SHA-256 + Serpent | Full-Disk Encryption (FDE)

29523 | LUKS v1 SHA-256 + Twofish | Full-Disk Encryption (FDE)

29531 | LUKS v1 SHA-512 + AES | Full-Disk Encryption (FDE)

29532 | LUKS v1 SHA-512 + Serpent | Full-Disk Encryption (FDE)

29533 | LUKS v1 SHA-512 + Twofish | Full-Disk Encryption (FDE)

13711 | VeraCrypt RIPEMD160 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

13712 | VeraCrypt RIPEMD160 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

13713 | VeraCrypt RIPEMD160 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

13741 | VeraCrypt RIPEMD160 + XTS 512 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13742 | VeraCrypt RIPEMD160 + XTS 1024 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13743 | VeraCrypt RIPEMD160 + XTS 1536 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

29411 | VeraCrypt RIPEMD160 + XTS 512 bit | Full-Disk Encryption (FDE)

29412 | VeraCrypt RIPEMD160 + XTS 1024 bit | Full-Disk Encryption (FDE)

29413 | VeraCrypt RIPEMD160 + XTS 1536 bit | Full-Disk Encryption (FDE)

29441 | VeraCrypt RIPEMD160 + XTS 512 bit + boot-mode | Full-Disk Encryption (FDE)

29442 | VeraCrypt RIPEMD160 + XTS 1024 bit + boot-mode | Full-Disk Encryption (FDE)

29443 | VeraCrypt RIPEMD160 + XTS 1536 bit + boot-mode | Full-Disk Encryption (FDE)

13751 | VeraCrypt SHA256 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

13752 | VeraCrypt SHA256 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

13753 | VeraCrypt SHA256 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

13761 | VeraCrypt SHA256 + XTS 512 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13762 | VeraCrypt SHA256 + XTS 1024 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13763 | VeraCrypt SHA256 + XTS 1536 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

29451 | VeraCrypt SHA256 + XTS 512 bit | Full-Disk Encryption (FDE)

29452 | VeraCrypt SHA256 + XTS 1024 bit | Full-Disk Encryption (FDE)

29453 | VeraCrypt SHA256 + XTS 1536 bit | Full-Disk Encryption (FDE)

29461 | VeraCrypt SHA256 + XTS 512 bit + boot-mode | Full-Disk Encryption (FDE)

29462 | VeraCrypt SHA256 + XTS 1024 bit + boot-mode | Full-Disk Encryption (FDE)

29463 | VeraCrypt SHA256 + XTS 1536 bit + boot-mode | Full-Disk Encryption (FDE)

13721 | VeraCrypt SHA512 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

13722 | VeraCrypt SHA512 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

13723 | VeraCrypt SHA512 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

29421 | VeraCrypt SHA512 + XTS 512 bit | Full-Disk Encryption (FDE)

29422 | VeraCrypt SHA512 + XTS 1024 bit | Full-Disk Encryption (FDE)

29423 | VeraCrypt SHA512 + XTS 1536 bit | Full-Disk Encryption (FDE)

13771 | VeraCrypt Streebog-512 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

13772 | VeraCrypt Streebog-512 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

13773 | VeraCrypt Streebog-512 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

13781 | VeraCrypt Streebog-512 + XTS 512 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13782 | VeraCrypt Streebog-512 + XTS 1024 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

13783 | VeraCrypt Streebog-512 + XTS 1536 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

29471 | VeraCrypt Streebog-512 + XTS 512 bit | Full-Disk Encryption (FDE)

29472 | VeraCrypt Streebog-512 + XTS 1024 bit | Full-Disk Encryption (FDE)

29473 | VeraCrypt Streebog-512 + XTS 1536 bit | Full-Disk Encryption (FDE)

29481 | VeraCrypt Streebog-512 + XTS 512 bit + boot-mode | Full-Disk Encryption (FDE)

29482 | VeraCrypt Streebog-512 + XTS 1024 bit + boot-mode | Full-Disk Encryption (FDE)

29483 | VeraCrypt Streebog-512 + XTS 1536 bit + boot-mode | Full-Disk Encryption (FDE)

13731 | VeraCrypt Whirlpool + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

13732 | VeraCrypt Whirlpool + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

13733 | VeraCrypt Whirlpool + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

29431 | VeraCrypt Whirlpool + XTS 512 bit | Full-Disk Encryption (FDE)

29432 | VeraCrypt Whirlpool + XTS 1024 bit | Full-Disk Encryption (FDE)

29433 | VeraCrypt Whirlpool + XTS 1536 bit | Full-Disk Encryption (FDE)

23900 | BestCrypt v3 Volume Encryption | Full-Disk Encryption (FDE)

16700 | FileVault 2 | Full-Disk Encryption (FDE)

27500 | VirtualBox (PBKDF2-HMAC-SHA256 & AES-128-XTS) | Full-Disk Encryption (FDE)

27600 | VirtualBox (PBKDF2-HMAC-SHA256 & AES-256-XTS) | Full-Disk Encryption (FDE)

20011 | DiskCryptor SHA512 + XTS 512 bit | Full-Disk Encryption (FDE)

20012 | DiskCryptor SHA512 + XTS 1024 bit | Full-Disk Encryption (FDE)

20013 | DiskCryptor SHA512 + XTS 1536 bit | Full-Disk Encryption (FDE)

22100 | BitLocker | Full-Disk Encryption (FDE)

12900 | Android FDE (Samsung DEK) | Full-Disk Encryption (FDE)

8800 | Android FDE <= 4.3 | Full-Disk Encryption (FDE)

18300 | Apple File System (APFS) | Full-Disk Encryption (FDE)

6211 | TrueCrypt RIPEMD160 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

6212 | TrueCrypt RIPEMD160 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

6213 | TrueCrypt RIPEMD160 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

6241 | TrueCrypt RIPEMD160 + XTS 512 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

6242 | TrueCrypt RIPEMD160 + XTS 1024 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

6243 | TrueCrypt RIPEMD160 + XTS 1536 bit + boot-mode (legacy) | Full-Disk Encryption (FDE)

29311 | TrueCrypt RIPEMD160 + XTS 512 bit | Full-Disk Encryption (FDE)

29312 | TrueCrypt RIPEMD160 + XTS 1024 bit | Full-Disk Encryption (FDE)

29313 | TrueCrypt RIPEMD160 + XTS 1536 bit | Full-Disk Encryption (FDE)

29341 | TrueCrypt RIPEMD160 + XTS 512 bit + boot-mode | Full-Disk Encryption (FDE)

29342 | TrueCrypt RIPEMD160 + XTS 1024 bit + boot-mode | Full-Disk Encryption (FDE)

29343 | TrueCrypt RIPEMD160 + XTS 1536 bit + boot-mode | Full-Disk Encryption (FDE)

6221 | TrueCrypt SHA512 + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

6222 | TrueCrypt SHA512 + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

6223 | TrueCrypt SHA512 + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

29321 | TrueCrypt SHA512 + XTS 512 bit | Full-Disk Encryption (FDE)

29322 | TrueCrypt SHA512 + XTS 1024 bit | Full-Disk Encryption (FDE)

29323 | TrueCrypt SHA512 + XTS 1536 bit | Full-Disk Encryption (FDE)

6231 | TrueCrypt Whirlpool + XTS 512 bit (legacy) | Full-Disk Encryption (FDE)

6232 | TrueCrypt Whirlpool + XTS 1024 bit (legacy) | Full-Disk Encryption (FDE)

6233 | TrueCrypt Whirlpool + XTS 1536 bit (legacy) | Full-Disk Encryption (FDE)

29331 | TrueCrypt Whirlpool + XTS 512 bit | Full-Disk Encryption (FDE)

29332 | TrueCrypt Whirlpool + XTS 1024 bit | Full-Disk Encryption (FDE)

29333 | TrueCrypt Whirlpool + XTS 1536 bit | Full-Disk Encryption (FDE)

12200 | eCryptfs | Full-Disk Encryption (FDE)

10400 | PDF 1.1 - 1.3 (Acrobat 2 - 4) | Document

10410 | PDF 1.1 - 1.3 (Acrobat 2 - 4), collider #1 | Document

10420 | PDF 1.1 - 1.3 (Acrobat 2 - 4), collider #2 | Document

10500 | PDF 1.4 - 1.6 (Acrobat 5 - 8) | Document

25400 | PDF 1.4 - 1.6 (Acrobat 5 - 8) - user and owner pass | Document

10600 | PDF 1.7 Level 3 (Acrobat 9) | Document

10700 | PDF 1.7 Level 8 (Acrobat 10 - 11) | Document

9400 | MS Office 2007 | Document

9500 | MS Office 2010 | Document

9600 | MS Office 2013 | Document

25300 | MS Office 2016 - SheetProtection | Document

9700 | MS Office <= 2003 $0/$1, MD5 + RC4 | Document

9710 | MS Office <= 2003 $0/$1, MD5 + RC4, collider #1 | Document

9720 | MS Office <= 2003 $0/$1, MD5 + RC4, collider #2 | Document

9810 | MS Office <= 2003 $3, SHA1 + RC4, collider #1 | Document

9820 | MS Office <= 2003 $3, SHA1 + RC4, collider #2 | Document

9800 | MS Office <= 2003 $3/$4, SHA1 + RC4 | Document

18400 | Open Document Format (ODF) 1.2 (SHA-256, AES) | Document

18600 | Open Document Format (ODF) 1.1 (SHA-1, Blowfish) | Document

16200 | Apple Secure Notes | Document

23300 | Apple iWork | Document

6600 | 1Password, agilekeychain | Password Manager

8200 | 1Password, cloudkeychain | Password Manager

9000 | Password Safe v2 | Password Manager

5200 | Password Safe v3 | Password Manager

6800 | LastPass + LastPass sniffed | Password Manager

13400 | KeePass 1 (AES/Twofish) and KeePass 2 (AES) | Password Manager

29700 | KeePass 1 (AES/Twofish) and KeePass 2 (AES) - keyfile only mode | Password Manager

23400 | Bitwarden | Password Manager

16900 | Ansible Vault | Password Manager

26000 | Mozilla key3.db | Password Manager

26100 | Mozilla key4.db | Password Manager

23100 | Apple Keychain | Password Manager

11600 | 7-Zip | Archive

12500 | RAR3-hp | Archive

23800 | RAR3-p (Compressed) | Archive

23700 | RAR3-p (Uncompressed) | Archive

13000 | RAR5 | Archive

17220 | PKZIP (Compressed Multi-File) | Archive

17200 | PKZIP (Compressed) | Archive

17225 | PKZIP (Mixed Multi-File) | Archive

17230 | PKZIP (Mixed Multi-File Checksum-Only) | Archive

17210 | PKZIP (Uncompressed) | Archive

20500 | PKZIP Master Key | Archive

20510 | PKZIP Master Key (6 byte optimization) | Archive

23001 | SecureZIP AES-128 | Archive

23002 | SecureZIP AES-192 | Archive

23003 | SecureZIP AES-256 | Archive

13600 | WinZip | Archive

18900 | Android Backup | Archive

24700 | Stuffit5 | Archive

13200 | AxCrypt 1 | Archive

13300 | AxCrypt 1 in-memory SHA1 | Archive

23500 | AxCrypt 2 AES-128 | Archive

23600 | AxCrypt 2 AES-256 | Archive

14700 | iTunes backup < 10.0 | Archive

14800 | iTunes backup >= 10.0 | Archive

8400 | WBB3 (Woltlab Burning Board) | Forums, CMS, E-Commerce

2612 | PHPS | Forums, CMS, E-Commerce

121 | SMF (Simple Machines Forum) > v1.1 | Forums, CMS, E-Commerce

3711 | MediaWiki B type | Forums, CMS, E-Commerce

4521 | Redmine | Forums, CMS, E-Commerce

24800 | Umbraco HMAC-SHA1 | Forums, CMS, E-Commerce

11 | Joomla < 2.5.18 | Forums, CMS, E-Commerce

13900 | OpenCart | Forums, CMS, E-Commerce

11000 | PrestaShop | Forums, CMS, E-Commerce

16000 | Tripcode | Forums, CMS, E-Commerce

7900 | Drupal7 | Forums, CMS, E-Commerce

4522 | PunBB | Forums, CMS, E-Commerce

2811 | MyBB 1.2+, IPB2+ (Invision Power Board) | Forums, CMS, E-Commerce

2611 | vBulletin < v3.8.5 | Forums, CMS, E-Commerce

2711 | vBulletin >= v3.8.5 | Forums, CMS, E-Commerce

25600 | bcrypt(md5($pass)) / bcryptmd5 | Forums, CMS, E-Commerce

25800 | bcrypt(sha1($pass)) / bcryptsha1 | Forums, CMS, E-Commerce

28400 | bcrypt(sha512($pass)) / bcryptsha512 | Forums, CMS, E-Commerce

21 | osCommerce, xt:Commerce | Forums, CMS, E-Commerce

18100 | TOTP (HMAC-SHA1) | One-Time Password

2000 | STDOUT | Plaintext

99999 | Plaintext | Plaintext

21600 | Web2py pbkdf2-sha512 | Framework

10000 | Django (PBKDF2-SHA256) | Framework

124 | Django (SHA-1) | Framework

12001 | Atlassian (PBKDF2-HMAC-SHA1) | Framework

19500 | Ruby on Rails Restful-Authentication | Framework

27200 | Ruby on Rails Restful Auth (one round, no sitekey) | Framework

30000 | Python Werkzeug MD5 (HMAC-MD5 (key = $salt)) | Framework

30120 | Python Werkzeug SHA256 (HMAC-SHA256 (key = $salt)) | Framework

20200 | Python passlib pbkdf2-sha512 | Framework

20300 | Python passlib pbkdf2-sha256 | Framework

20400 | Python passlib pbkdf2-sha1 | Framework

24410 | PKCS#8 Private Keys (PBKDF2-HMAC-SHA1 + 3DES/AES) | Private Key

24420 | PKCS#8 Private Keys (PBKDF2-HMAC-SHA256 + 3DES/AES) | Private Key

15500 | JKS Java Key Store Private Keys (SHA1) | Private Key

22911 | RSA/DSA/EC/OpenSSH Private Keys (0) | Private Key

22921 | RSA/DSA/EC/OpenSSH Private Keys (6) | Private Key

22931 | RSA/DSA/EC/OpenSSH Private Keys ($1, 3) | Private Key

22941 | RSA/DSA/EC/OpenSSH Private Keys (4) | Private Key

22951 | RSA/DSA/EC/OpenSSH Private Keys (5) | Private Key

23200 | XMPP SCRAM PBKDF2-SHA1 | Instant Messaging Service

28300 | Teamspeak 3 (channel hash) | Instant Messaging Service

22600 | Telegram Desktop < v2.1.14 (PBKDF2-HMAC-SHA1) | Instant Messaging Service

24500 | Telegram Desktop >= v2.1.14 (PBKDF2-HMAC-SHA512) | Instant Messaging Service

22301 | Telegram Mobile App Passcode (SHA256) | Instant Messaging Service

23 | Skype | Instant Messaging Service

29600 | Terra Station Wallet (AES256-CBC(PBKDF2($pass))) | Cryptocurrency Wallet

26600 | MetaMask Wallet | Cryptocurrency Wallet

21000 | BitShares v0.x - sha512(sha512_bin(pass)) | Cryptocurrency Wallet

28501 | Bitcoin WIF private key (P2PKH), compressed | Cryptocurrency Wallet

28502 | Bitcoin WIF private key (P2PKH), uncompressed | Cryptocurrency Wallet

28503 | Bitcoin WIF private key (P2WPKH, Bech32), compressed | Cryptocurrency Wallet

28504 | Bitcoin WIF private key (P2WPKH, Bech32), uncompressed | Cryptocurrency Wallet

28505 | Bitcoin WIF private key (P2SH(P2WPKH)), compressed | Cryptocurrency Wallet

28506 | Bitcoin WIF private key (P2SH(P2WPKH)), uncompressed | Cryptocurrency Wallet

11300 | Bitcoin/Litecoin wallet.dat | Cryptocurrency Wallet

16600 | Electrum Wallet (Salt-Type 1-3) | Cryptocurrency Wallet

21700 | Electrum Wallet (Salt-Type 4) | Cryptocurrency Wallet

21800 | Electrum Wallet (Salt-Type 5) | Cryptocurrency Wallet

12700 | Blockchain, My Wallet | Cryptocurrency Wallet

15200 | Blockchain, My Wallet, V2 | Cryptocurrency Wallet

18800 | Blockchain, My Wallet, Second Password (SHA256) | Cryptocurrency Wallet

25500 | Stargazer Stellar Wallet XLM | Cryptocurrency Wallet

16300 | Ethereum Pre-Sale Wallet, PBKDF2-HMAC-SHA256 | Cryptocurrency Wallet

15600 | Ethereum Wallet, PBKDF2-HMAC-SHA256 | Cryptocurrency Wallet

15700 | Ethereum Wallet, SCRYPT | Cryptocurrency Wallet

22500 | MultiBit Classic .key (MD5) | Cryptocurrency Wallet

27700 | MultiBit Classic .wallet (scrypt) | Cryptocurrency Wallet

22700 | MultiBit HD (scrypt) | Cryptocurrency Wallet

28200 | Exodus Desktop Wallet (scrypt) | Cryptocurrency Wallet

- [Brain Client Features] -

#	Features
1	Send hashed passwords
2	Send attack positions
3	Send hashed passwords and attack positions

- [Outfile Formats] -

#	Format
1	hash[:salt]
2	plain
3	hex_plain
4	crack_pos
5	timestamp absolute
6	timestamp relative

- [Rule Debugging Modes] -

#	Format
1	Finding-Rule
2	Original-Word
3	Original-Word:Finding-Rule
4	Original-Word:Finding-Rule:Processed-Word
5	Original-Word:Finding-Rule:Processed-Word:Wordlist

- [Attack Modes] -

 # | Mode

===+======

 0 | Straight

 1 | Combination

 3 | Brute-force

 6 | Hybrid Wordlist + Mask

 7 | Hybrid Mask + Wordlist

 9 | Association

- [Built-in Charsets] -

 ? | Charset

===+=========

 l | abcdefghijklmnopqrstuvwxyz [a-z]

 u | ABCDEFGHIJKLMNOPQRSTUVWXYZ [A-Z]

 d | 0123456789 [0-9]

 h | 0123456789abcdef [0-9a-f]

 H | 0123456789ABCDEF [0-9A-F]

 s | !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~

 a | ?l?u?d?s

 b | 0x00 - 0xff

- [OpenCL Device Types] -

 # | Device Type

===+=============

1 | CPU

2 | GPU

3 | FPGA, DSP, Co-Processor

- [Workload Profiles] -

| Performance | Runtime | Power Consumption | Desktop Impact

===+=============+=========+===================+================

1 | Low | 2 ms | Low | Minimal

2 | Default | 12 ms | Economic | Noticeable

3 | High | 96 ms | High | Unresponsive

4 | Nightmare | 480 ms | Insane | Headless

- [License] -

hashcat is licensed under the MIT license

Copyright and license terms are listed in docs/license.txt

- [Basic Examples] -

Attack- | Hash- |

Mode | Type | Example command

==========+=======+===

=========

Wordlist | P | hashcat -a 0 -m 400 example400.hash example.dict

Wordlist + Rules | MD5 | hashcat -a 0 -m 0 example0.hash example.dict -r rules/best64.rule

Brute-Force | MD5 | hashcat -a 3 -m 0 example0.hash ?a?a?a?a?a?a

Combinator | MD5 | hashcat -a 1 -m 0 example0.hash example.dict example.dict

Association | 1 | hashcat -a 9 -m 500 example500.hash 1word.dict -r rules/best64.rule

If you still have no idea what just happened, try the following pages:

* https://hashcat.net/wiki/#howtos_videos_papers_articles_etc_in_the_wild

* https://hashcat.net/faq/